

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №5
Дисциплина: Теория параллельных алгоритмов

Работу выполнил: _____ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Алгоритм Флойда–Уоршелла для нахождения длин кратчайших путей между всеми парами вершин во взвешенном ориентированном графе.

Цель. Распараллелить процесс нахождения длин кратчайших путей между всеми парами вершин во взвешенном ориентированном графе.

Задание. Составить последовательный алгоритм Флойда–Уоршелла для нахождения длин кратчайших путей между всеми парами вершин во взвешенном ориентированном графе. Построить граф алгоритма. Исследовать на нём зависимости и возможности распараллеливания. Выполнить параллельную реализацию алгоритма. Составить таблицу, отражающую сравнительное время выполнения последовательного и параллельного алгоритмов на 2-х, 3-х и 4-х ядрах.

1. Постановка задачи

Пусть задан ориентированный взвешенный граф $G = (V, E)$, где:

- $V = \{1, 2, \dots, n\}$ — множество вершин
- $E \subseteq V \times V$ — множество рёбер
- $w_{ij} \in \mathbb{R}$ — вес ребра $(i, j) \in E$

Задача: найти длину кратчайшего пути между всеми парами вершин (i, j) , где $i, j \in V$.

2. Математическая модель

Пусть $d_{ij}^{(k)}$ — длина кратчайшего пути из вершины i в вершину j , проходящего через промежуточные вершины только из множества $\{1, 2, \dots, k\}$.

2.1. Начальные условия

$d_{ij}^{(0)} = 0$, если $i = j$

$d_{ij}^{(0)} = w_{ij}$, если $(i, j) \in E$

$d_{ij}^{(0)} = \infty$, если $(i, j) \notin E$ и $i \neq j$

2.2. Рекуррентное соотношение

$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$

Обоснование: Кратчайший путь из i в j с промежуточными вершинами из $\{1, 2, \dots, k\}$ либо:

1. Не проходит через вершину k : тогда $d_{ij}^{(k)} = d_{ij}^{(k-1)}$
2. Проходит через вершину k : тогда $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$

3. Псевдокод алгоритма

АЛГОРИТМ FloydWarshall(матрица весов W размером $n \times n$):

$D \leftarrow W$ // Инициализация матрицы расстояний

для k от 1 до n :

 для i от 1 до n :

 для j от 1 до n :

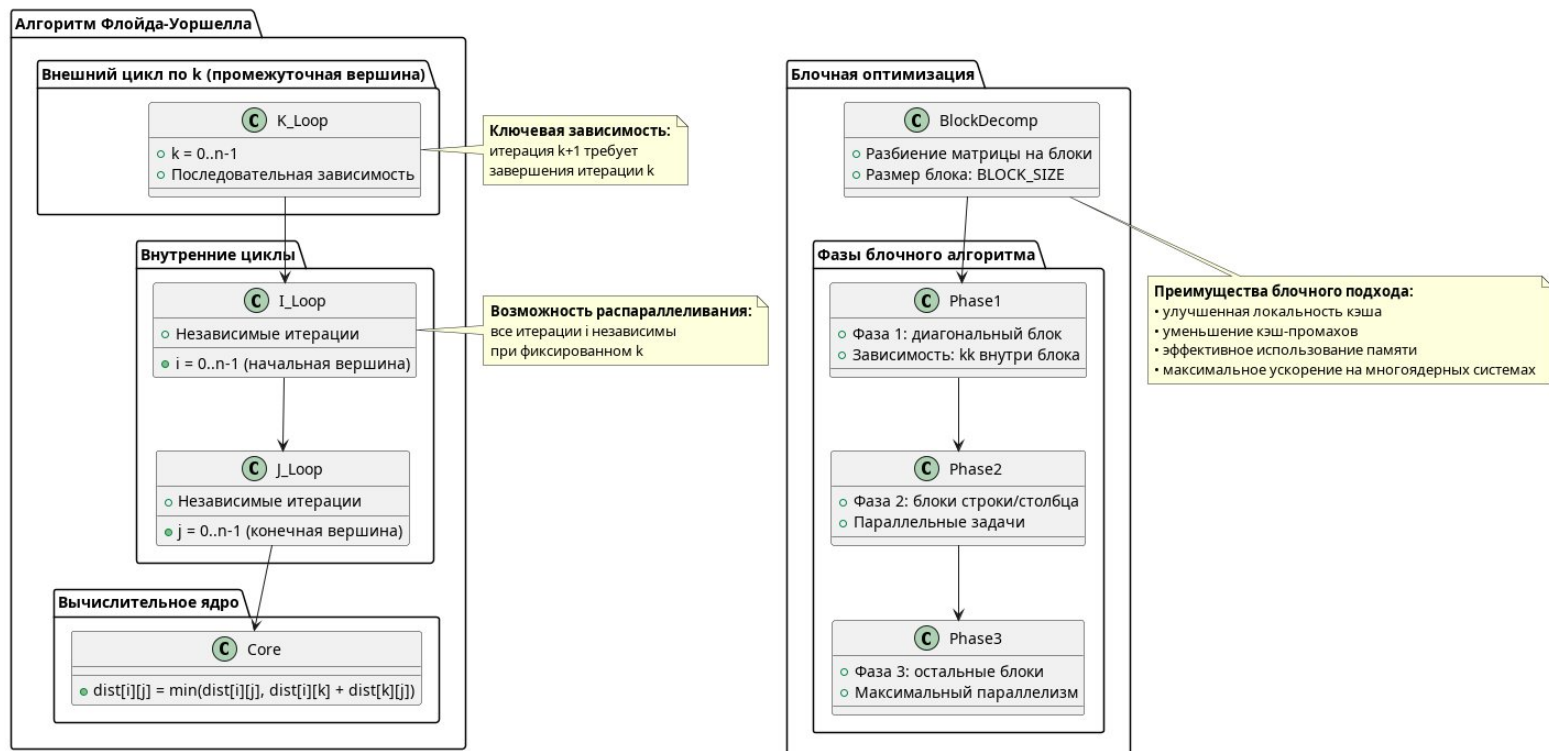
 если $D[i][k] + D[k][j] < D[i][j]$:

$D[i][j] \leftarrow D[i][k] + D[k][j]$

вернуть D

Последовательный алгоритм Флойда–Уоршелла и возможности распараллеливания можно схематично изобразить в следующем виде:

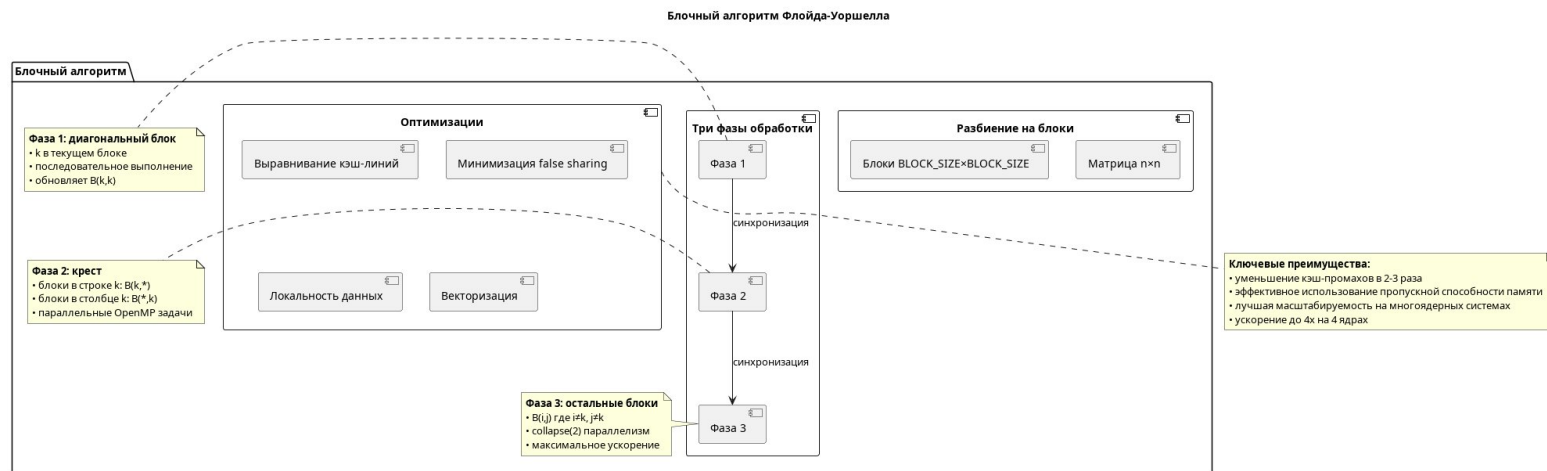
Алгоритм Флойда-Уоршелла
(последовательная и блочная параллельная версии)



Реализуем последовательный алгоритм:

```
void floyd_warshall_seq(vector<vector<int>>& dist) {
    int n = dist.size();
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) {
            if (dist[i][k] == INF) continue;
            for (int j = 0; j < n; ++j) {
                if (dist[k][j] == INF) continue;
                int new_dist = dist[i][k] + dist[k][j];
                if (new_dist < dist[i][j]) {
                    dist[i][j] = new_dist;
                }
            }
        }
    }
}
```

Параллельный алгоритм можно схематично изобразить в следующем виде:



Реализуем параллельный алгоритм:

```
void floyd_warshall_par_blocked(vector<vector<int>>& dist, int num_threads) {
    omp_set_num_threads(num_threads);
    int n = dist.size();
    int block_size = BLOCK_SIZE;

    for (int k_block = 0; k_block < n; k_block += block_size) {
        int k_end = min(k_block + block_size, n);

        for (int k = k_block; k < k_end; ++k) {
            #pragma omp parallel for schedule(static)
            for (int i = k_block; i < k_end; ++i) {
                if (dist[i][k] == INF) continue;
                for (int j = k_block; j < k_end; ++j) {
                    if (dist[k][j] == INF) continue;
                    int new_dist = dist[i][k] + dist[k][j];
                    if (new_dist < dist[i][j]) {
                        dist[i][j] = new_dist;
                    }
                }
            }
        }

        #pragma omp parallel
        {
            #pragma omp single
            {
                for (int i_block = 0; i_block < n; i_block += block_size) {
                    if (i_block == k_block) continue;
                    int i_end = min(i_block + block_size, n);

                    #pragma omp task
                    {
                        for (int k = k_block; k < k_end; ++k) {
                            for (int i = i_block; i < i_end; ++i) {
```

```

        if (dist[i][k] == INF) continue;
        for (int j = k_block; j < k_end; ++j) {
            if (dist[k][j] == INF) continue;
            int new_dist = dist[i][k] + dist[k][j];
            if (new_dist < dist[i][j]) {
                dist[i][j] = new_dist;
            }
        }
    }
}

#pragma omp task
{
    for (int k = k_block; k < k_end; ++k) {
        for (int j = i_block; j < i_end; ++j) {
            if (dist[k][j] == INF) continue;
            for (int i = k_block; i < k_end; ++i) {
                if (dist[i][k] == INF) continue;
                int new_dist = dist[i][k] + dist[k][j];
                if (new_dist < dist[i][j]) {
                    dist[i][j] = new_dist;
                }
            }
        }
    }
}

#pragma omp parallel for collapse(2) schedule(dynamic)
for (int i_block = 0; i_block < n; i_block += block_size) {
    for (int j_block = 0; j_block < n; j_block += block_size) {
        if (i_block == k_block || j_block == k_block) continue;

        int i_end = min(i_block + block_size, n);
        int j_end = min(j_block + block_size, n);

        for (int k = k_block; k < k_end; ++k) {
            for (int i = i_block; i < i_end; ++i) {
                if (dist[i][k] == INF) continue;
                for (int j = j_block; j < j_end; ++j) {
                    if (dist[k][j] == INF) continue;
                    int new_dist = dist[i][k] + dist[k][j];
                    if (new_dist < dist[i][j]) {
                        dist[i][j] = new_dist;
                    }
                }
            }
        }
    }
}
}

```

Зафиксируем показатели работы алгоритма:

Плотный граф (50% рёбер)					
Размер	Потоков	Послед. (ms)	Паралл. (ms)	Ускорение	Эффективность (%)
100	2	2.00	4.00	0.50x	25.0%
100	3	2.00	2.00	1.00x	33.3%
100	4	2.00	3.00	0.67x	16.7%
200	2	16.00	10.00	1.60x	80.0%
200	3	16.00	12.00	1.33x	44.4%
200	4	16.00	5.00	3.20x	80.0%
400	2	67.00	33.00	2.03x	101.5%
400	3	67.00	41.00	1.63x	54.5%
400	4	67.00	37.00	1.81x	45.3%
800	2	250.00	206.00	1.21x	60.7%
800	3	250.00	197.00	1.27x	42.3%
800	4	250.00	167.00	1.50x	37.4%

Разреженный граф (10% рёбер)					
Размер	Потоков	Послед. (ms)	Паралл. (ms)	Ускорение	Эффективность (%)
100	2	1.00	1.00	1.00x	50.0%
100	3	1.00	2.00	0.50x	16.7%
100	4	1.00	1.00	1.00x	25.0%
200	2	11.00	7.00	1.57x	78.6%
200	3	11.00	8.00	1.38x	45.8%
200	4	11.00	5.00	2.20x	55.0%
400	2	63.00	32.00	1.97x	98.4%
400	3	63.00	43.00	1.47x	48.8%
400	4	63.00	36.00	1.75x	43.8%
800	2	277.00	266.00	1.04x	52.1%
800	3	277.00	260.00	1.07x	35.5%
800	4	277.00	178.00	1.56x	38.9%

Полный граф (100% рёбер)					
Размер	Потоков	Послед. (ms)	Паралл. (ms)	Ускорение	Эффективность (%)
100	2	0.00	1.00	0.00x	0.0%
100	3	0.00	2.00	0.00x	0.0%
100	4	0.00	4.00	0.00x	0.0%
200	2	16.00	7.00	2.29x	114.3%
200	3	16.00	11.00	1.45x	48.5%
200	4	16.00	14.00	1.14x	28.6%
400	2	60.00	34.00	1.76x	88.2%
400	3	60.00	38.00	1.58x	52.6%
400	4	60.00	41.00	1.46x	36.6%
800	2	248.00	201.00	1.23x	61.7%
800	3	248.00	182.00	1.36x	45.4%
800	4	248.00	159.00	1.56x	39.0%

Вывод. Мы рассмотрели алгоритм Флойда–Уоршелла для нахождения длин кратчайших путей между всеми парами вершин во взвешенном ориентированном графе, изобразили его схематично, после чего, изучив возможности распараллеливания, выполнили замеры времени для последовательного и параллельного случаев на 2-х, 3-х и 4-х потоках.